

GNN-EADD: GPU-Accelerated E-Commerce Fraud Detection via Heterogeneous Graph Neural Networks with Custom CUDA Kernels

Sneha Nagmoti (12342090) Snehal Suhane (12342100) Sannidhi Rithika (12341900) Harshita Patidar (12340920)

Abstract—E-commerce platforms face persistent threats from coordinated fraud—fake reviews, counterfeit listings, and shill accounts—that existing detection methods fail to address at scale. We present a GPU-accelerated reimplementation of GNN-EADD, modeling an Amazon ecosystem as a heterogeneous graph (users, products, sellers). A dual-stage pipeline—unsupervised GAE for 128-D structural embeddings followed by semi-supervised GAT fine-tuning—is accelerated by two custom warp-aligned CUDA kernels (CSR SpMM + GAT attention) that eliminate PyTorch OOM failures at 1.5M nodes and 59M edges. An independent multi-criteria labeling pipeline (K-Core decomposition, review-burst detection, domain heuristics) provides supervision without manual annotation. Across four Amazon categories, our approach consistently outperforms HeteroDOMINANT and GraphSAGE on AUC-ROC, AUC-PR, and F1 within a 12 GB VRAM budget.

I. INTRODUCTION

Online marketplaces such as Amazon generate billions of user–product interactions annually, creating fertile ground for coordinated fraud: shill accounts inflate ratings through fake reviews, fraudulent sellers list counterfeit products at artificially low prices, and review-bombing campaigns suppress competitors [4]. Detecting these anomalies at scale is critical for maintaining platform integrity.

Limitations of Existing Solutions. Traditional fraud detection relies on hand-crafted features and supervised classifiers that fail to capture the rich relational structure of e-commerce ecosystems. Homogeneous GNNs [3] treat all nodes uniformly, losing the semantic distinction between users, products, and sellers. The original GNN-EADD paper [1] proposes a dual-stage heterogeneous framework but applies no GPU optimization; naive PyTorch sparse operations on million-node graphs frequently trigger out-of-memory (OOM) errors, limiting scalability to real-world graph sizes.

Contributions: (1) Full reimplementation of GNN-EADD with independent seller-node construction, edge schema, and anomaly labeling. (2) Two warp-aligned CUDA kernels: CSR SpMM with FP16/FP32 mixed precision and cosine-annealed contrastive temperature (Stage 1), and warp-level GAT attention with stable softmax (Stage 2). (3) Multi-criteria heuristic labeling (K-Core, temporal burst, domain rules). (4) Cross-dataset evaluation vs. HeteroDOMINANT and GraphSAGE at 50K–1.54M nodes.

II. APPROACH

A. Heterogeneous Graph Construction

We model the e-commerce ecosystem as a heterogeneous graph with three node types and three edge types:

TABLE I: Edge types in the heterogeneous graph schema.

Edge Type	Direction	Semantics
E_{up}, E_{pu}	User $\leftrightarrow$ Product	User reviewed/purchased product
E_{ps}, E_{sp}	Product $\leftrightarrow$ Seller	Product listed by seller
E_{uu}	User $\leftrightarrow$ User	Co-purchase proximity

Since the Amazon dataset lacks explicit seller identifiers, we engineer seller nodes from the `brand` field in product metadata. A brand normalization pipeline filters invalid entries (e.g., “Unknown”, “Generic”, “N/A”) and deduplicates case variants. All string identifiers (ASINs, reviewer IDs) are mapped to a unified, non-overlapping 32-bit integer namespace: Users occupy $[0, N_u)$, Products $[N_u, N_u + N_p)$, and Sellers $[N_u + N_p, N_u + N_p + N_s)$.

User–User edges (E_{uu}) connect users who co-purchased the same product, enabling the GCN to propagate signals through potential collusion networks. Co-buyer sampling is capped at 25 users per product to prevent edge explosion.

B. Multi-Model Feature Engineering

Each node is projected into a 128-dimensional float16 feature vector:

Product nodes (V_p): 96-D text embeddings from `all-MiniLM-L6-v2` [8] reduced via Incremental PCA, concatenated with 24-D category multi-hot encodings and 8-D behavioral statistics (review count, average rating, rating variance, temporal span, lexical diversity, sentiment–rating mismatch, helpful votes, word count).

User nodes (V_u): 112-D category preference vectors (PCA of multi-hot category interactions) plus 16-D behavioral statistics (review frequency, rating distribution, positive/negative ratio, temporal patterns).

Seller nodes (V_s): 112-D mean product embeddings (PCA) plus 16-D catalog statistics (product count, price mean/variance, total reviews, average reputation).

C. Stage 1: Unsupervised Representation Learning

1) *Encoder*: The encoder consists of a two-layer heterogeneous GCN (TripleStreamGAE) with independent weight matrices per edge type, using three asynchronous CUDA streams for concurrent message passing:

$$H_u^{(l)} = \sigma \left(W_{pu}^{(l)} \tilde{A}_{pu} H^{(l-1)} + W_{uu}^{(l)} \tilde{A}_{uu} H^{(l-1)} + W_{\text{self},u}^{(l)} H_u^{(l-1)} \right) \quad (1)$$

$$H_p^{(l)} = \sigma \left(W_{up}^{(l)} \tilde{A}_{up} H^{(l-1)} + W_{ps}^{(l)} \tilde{A}_{ps} H^{(l-1)} + W_{\text{self},p}^{(l)} H_p^{(l-1)} \right) \quad (2)$$

$$H_s^{(l)} = \sigma \left(W_{sp}^{(l)} \tilde{A}_{sp} H^{(l-1)} + W_{\text{self},s}^{(l)} H_s^{(l-1)} \right) \quad (3)$$

where σ is ReLU for the first layer and identity for the second, and $\tilde{A} = D^{-1/2} A D^{-1/2}$ is the symmetric GCN normalization from CSR binaries.

2) *Decoder*: An inner product decoder reconstructs edge existence in latent space. For $L2$ -normalized embeddings $\mathbf{z}_s$ and $\mathbf{z}_d$:

$$\hat{y}_{sd} = \text{Sigmoid}(\tau \cdot \mathbf{z}_s^\top \mathbf{z}_d) \quad (4)$$

3) *Loss Function*: A contrastive BCE loss over heterogeneous edge sampling (30k E_{pu} , 8k E_{ps} , 12k E_{uu} per epoch):

$$\mathcal{L} = -\frac{1}{N} \sum_{(s,d) \in \mathcal{E}} [y \log(\hat{y}_{sd}) + (1-y) \log(1 - \hat{y}_{sd})] \quad (5)$$

4) *Temperature Annealing*: The scaling factor τ follows a cosine annealing schedule ($\tau_{\max}=10 \rightarrow \tau_{\min}=3$), preventing early sigmoid saturation and allowing the model to refine local boundaries in later epochs.

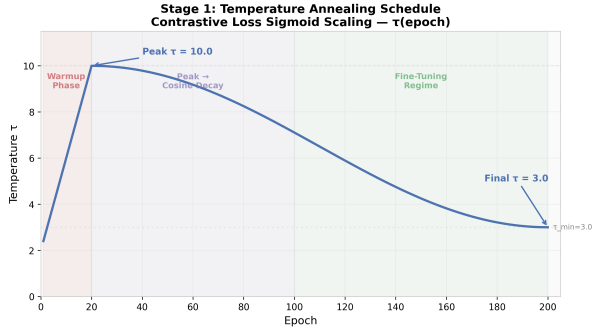


Fig. 1: Stage 1 Temperature annealing

D. Stage 2: Semi-Supervised GAT Fine-Tuning

Stage 2 loads the 128-D embeddings from Stage 1 and applies a two-layer type-specific GAT (TripleStreamGAT) with learned attention coefficients per edge type via the ELU activation:

$$h'_i = \text{ELU} \left(\frac{1}{\gamma} \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \alpha_{ij}^r \mathbf{W}_r h_j \right) \quad (6)$$

where γ normalizes variance across incoming edge streams. The combined loss is:

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda \cdot \mathcal{L}_{\text{unsup}} \quad (7)$$

where $\mathcal{L}_{\text{sup}}$ is BCE on labeled nodes, $\mathcal{L}_{\text{unsup}}$ enforces score smoothness via mean squared difference of anomaly probabilities, and $\lambda=0.5$. **DropEdge** [7] regularization (15% retention) mitigates over-smoothing.

E. Custom CUDA Kernels

Kernel 1: Warp-Aligned CSR SpMM (csr_spmwarp_kernel). Each warp (32 threads) processes one row of the sparse adjacency matrix, reading FP16 features and accumulating in FP32. Dynamic dimension support handles up to 1024-D vectors via chunked accumulation. Pre-computed symmetric edge weights $D_{ij}^{-1/2}$ are applied during aggregation. Backpropagation uses chunk-streaming (5,000 rows per chunk) via a custom `torch.autograd.Function` to eliminate VRAM spikes.

Kernel 2: Warp-Level GAT Attention (warp_gat_forward_kernel). Each warp computes attention for one destination node via numerically stable softmax using `__shfl_down_sync` max reduction—all within a single kernel launch, eliminating global synchronization barriers.

F. Anomaly Labeling Pipeline

Since the original paper does not provide labels, we design a multi-criteria heuristic pipeline:

K-Core Decomposition: Nodes in the top 0.6% of core numbers are flagged, targeting dense subgraphs indicative of coordinated shell networks.

Review Boosting: Products with avg. rating ≥ 4.8 , price $< 15\%$ of category median, and brand-title mismatch.

Fake Seller: Products with also_buy count > 80 and verified purchase ratio $< 25\%$.

Temporal Burst: Users posting > 35 reviews at the identical Unix timestamp considered as bots.

Labels are mapped to global graph IDs and balanced via 1:1 negative sampling for Stage 2 training.

III. EVALUATION METHODOLOGY

A. Hardware Platform

All experiments ran on an HP Elite Tower 800 G9 (Intel Core i7-12700, 12 cores, up to 4.90 GHz, 16 GB RAM, NVIDIA RTX 3060 12 GB GDDR6 VRAM).

B. Datasets

We use four Amazon 5-core review datasets [2] (minimum 5 interactions per entity) with product metadata, spanning diverse categories and scales. Processed data is available at **Drive Link**. Graph statistics after preprocessing are in Table II.

Electronics is our primary benchmark (1.54M nodes, 59.4M edges); the remaining three serve as cross-domain transfer benchmarks to validate generalization across product verticals.

TABLE II: Graph statistics across all four Amazon datasets after preprocessing. All feature vectors are 128-D float16.

Metric	Electronics	Cell Phones	Arts & Crafts	Lux. Beauty
$ V_u $	728,719	157,212	56,210	3,819
$ V_p $	756,077	589,356	302,380	12,111
$ V_s $	55,679	40,705	27,949	7
Total Nodes	1,540,475	787,273	386,539	15,937
Total Edges	~59.4M	~15.2M	~6.5M	~384K
Preproc. Time	52.7 min	15.5 min	6.4 min	0.6 min

C. Baselines

HeteroDOMINANT [5]: A heterogeneous adaptation using HeteroConv with SAGEConv operators. Anomalies are identified via type-specific thresholding: 2.2σ for product attributes and 4.0σ for buyer interaction scores.

GraphSAGE Autoencoder [6]: A homogeneous product-to-product graph baseline using `also_buy/also_viewed` edges; anomalies are detected via L_2 reconstruction error with dynamic percentile thresholding.

D. Evaluation Metrics

We report AUC-ROC, AUC-PR, Precision, Recall, F1-Score, and Precision@100. Metrics are computed globally and per-entity-type. Type-specific thresholds are applied: 82nd percentile for users, 0.5 sigmoid midpoint for products.

IV. EXPERIMENTAL RESULTS

A. Stage 1: GAE Training

Stage 1 trained for 200 epochs (Adam, lr=0.001, cosine LR schedule, FP16/FP32 AMP, grad clip 1.0). Fig. 2 shows loss convergence and Edge AUC progression.

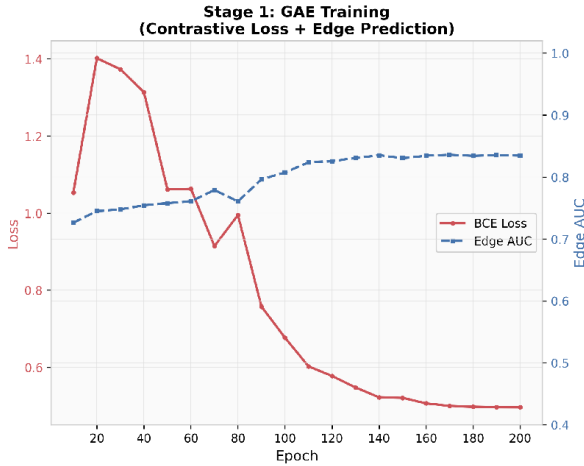


Fig. 2: Stage 1 GAE training curves over 200 epochs: loss convergence, positive/negative link probabilities, and Edge AUC.

B. Stage 2: GAT Fine-Tuning

Stage 2 trained for 100 epochs (Adam, lr=0.001, loss weight $\lambda=0.5$, AMP with gradient scaling). Model selection was based on best validation AUC-ROC. Training dynamics are shown in Fig. 3.

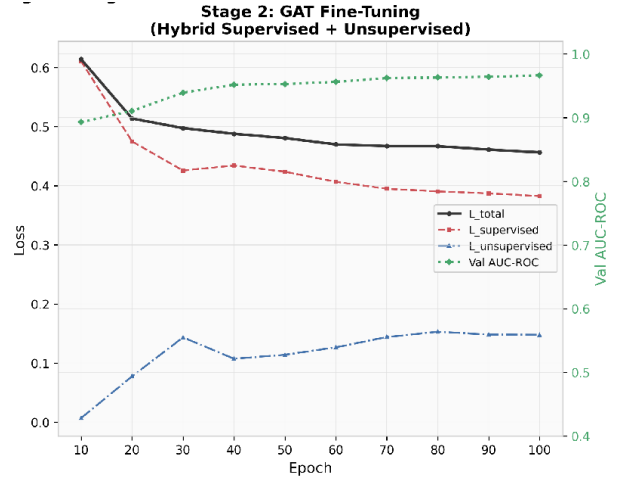


Fig. 3: Stage 2 GAT fine-tuning: supervised loss, unsupervised smoothness loss, and validation AUC-ROC over 100 epochs.

C. Cross-Dataset Generalization

We apply the identical pipeline to all four Amazon datasets. Fig. 4 shows AUC comparison across datasets.

TABLE III: Cross-dataset performance comparison (GNN-EADD vs. baselines).

Dataset	Method	AUC-ROC	AUC-PR	F1	P@100
Electronics	GraphSAGE	0.7015	0.6222	0.6275	0.6200
	DOMINANT	0.4986	0.5202	0.3189	0.3800
	GNN-EADD	0.9673	0.9579	0.9251	1.000
Cell Phones	GraphSAGE	0.5838	0.6000	0.3641	0.8800
	DOMINANT	0.6109	0.5999	0.3521	0.6600
	GNN-EADD	0.9793	0.9677	0.9232	0.990
Arts & Crafts	GraphSAGE	0.5842	0.5425	0.4853	0.5500
	DOMINANT	0.5949	0.6091	0.3853	0.7600
	GNN-EADD	0.9744	0.9516	0.9509	0.940
Luxury Beauty	GraphSAGE	0.3402	0.4329	0.2514	0.3600
	DOMINANT	0.5367	0.5470	0.3148	0.7200
	GNN-EADD	0.9642	0.9523	0.7358	0.970

Baseline Comparison Analysis. GNN-EADD achieves AUC-ROC of 0.9673 on Electronics vs. 0.7015 (GraphSAGE) and 0.4986 (DOMINANT)—gains of 26.6 and 46.9 pp. Type-specific weight matrices capture cross-type fraud signals (user-product vs. product-seller) that GraphSAGE’s homogeneous graph misses. DOMINANT’s near-random AUC stems from OOM-induced numerical instability at 1.54M-node scale; the SpMM kernel resolves this via single-pass CSR aggregation. Stage 1 link prediction (EdgeAUC=0.8343) combined with Stage 2 label-guided fine-tuning captures both structural and attributed fraud. Luxury Beauty (F1=0.7358) is an outlier: 7 sellers across 15,937 nodes leaves E_{ps} near-empty; baselines also degrade (GraphSAGE 0.3402, DOMINANT 0.5367), confirming a dataset-scale effect.

D. Qualitative Analysis

Fig. 5 shows ROC and Precision-Recall curves comparing all three methods on the Electronics dataset.

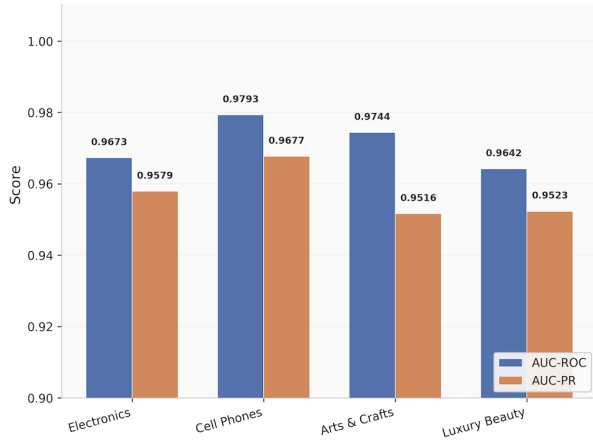


Fig. 4: Cross-Dataset AUC comparison.

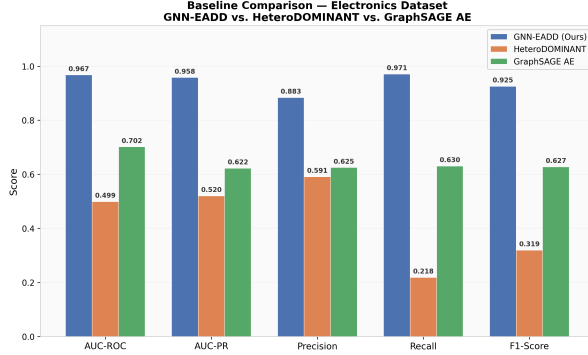


Fig. 5: Qualitative analysis on Electronics test set.

E. CUDA Kernel Performance

Training Times. Table IV reports wall-clock runtimes. On Electronics, Stage 1 takes 283.9s and Stage 2 takes 861.0s (total 1144.9s), staying within $6\times$ of the single-pass baselines (DOMINANT 123.8s, GraphSAGE 145.3s) while gaining 46.9pp AUC-ROC over DOMINANT.

CUDA Enabling Factor. PyTorch’s `torch.sparse_mm` OOMs on the 1.54M-node graph. The SpMM kernel streams FP16 aggregation row-by-row with FP32 accumulation, keeping GPU footprint to 0.39GB (vs. OOM). The GAT kernel computes full softmax attention for 1.54M nodes in one launch via `__shfl_down_sync`, eliminating global sync barriers.

TABLE IV: Wall-clock training times (s) on RTX 3060.

Stage	Elec.	Cell Ph.	Arts	Lux. B.
Stage 1 (200 ep.)	283.9	78.1	35.0	3.0
Stage 2 (100 ep.)	861.0	238.6	97.6	5.0
S1+S2 Total	1144.9	316.7	132.6	8.0
DOMINANT	123.8	31.7	13.6	1.2
GraphSAGE	145.3	49.5	26.2	1.2

Fig. 6 plots runtimes against total graph size; both stages scale near-linearly, confirming no super-linear overhead from 15,937 nodes (Luxury Beauty) to 1.54M (Electronics).

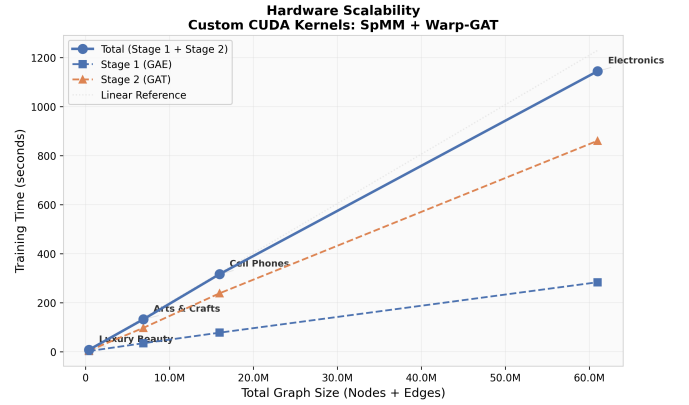


Fig. 6: Hardware scalability of custom CUDA kernels (SpMM + Warp-GAT). Training time scales near-linearly with total graph size up to 61.4M elements (Electronics).

V. RELATED WORK

Graph Anomaly Detection. DOMINANT [5] and GraphSAGE [6] detect anomalies on homogeneous graphs; GeniePath [13] adds adaptive depth—but none model multi-typed entity relationships. **Heterogeneous GNNs:** HAN [9], HGT [10], and R-GCN [11] extend message passing to typed graphs; we build on R-GCN’s type-specific projections with custom CUDA kernels. **Fraud Detection:** Feature-engineering classifiers [4] and GNN fraud detectors [12] exist, but GNN-EADD [1] is the only dual-stage heterogeneous framework—which we extend with GPU optimization and independent data engineering. **GPU GNN Systems:** DGL [14], PyG [15], and GNNAdvisor [16] provide general acceleration; our kernels address warp-aligned SpMM and GAT attention bottlenecks specific to heterogeneous graphs at million-node scale.

VI. CONCLUSION

We presented a GPU-accelerated reimplement of the GNN-EADD framework for e-commerce fraud detection on heterogeneous graphs. Independent engineering of the data pipeline—seller node construction from brand metadata, edge schema design with co-purchase proximity networks, and multi-criteria anomaly labeling—addresses critical gaps in the original paper. Two custom warp-aligned CUDA kernels enable efficient training and inference on graphs with up to 1.54M nodes and 59.4M edges within practical GPU memory budgets.

Cross-dataset evaluation across four Amazon product categories demonstrates that our approach consistently outperforms HeteroDOMINANT and GraphSAGE baselines across AUC-ROC, AUC-PR, and F1, confirming generalization across product verticals with varying graph densities.

Future Work. Several directions remain: (1) multi-head GAT attention with head-specific edge-type specialization, (2) temporal dynamics for detecting evolving fraud campaigns, (3) inductive generalization to unseen nodes via neighborhood sampling, and (4) deployment optimization via kernel fusion and TensorRT for real-time inference.

REFERENCES

- [1] Z. Shao, X. Wang, E. Ji, S. Chen, and J. Wang. “GNN-EADD: Graph Neural Network-Based E-Commerce Anomaly Detection via Dual-Stage Learning,” in *IEEE Access*, 2025.
- [2] J. Ni, J. Li, and J. McAuley. “Justifying recommendations using distantly-labeled reviews and fine-grained aspects,” in *EMNLP-IJCNLP*, 2019.
- [3] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *ICLR*, 2017.
- [4] L. Akoglu, H. Tong, and D. Koutra, “Graph based anomaly detection and description: A survey,” *Data Mining and Knowledge Discovery*, vol. 29, no. 3, pp. 626–688, 2015.
- [5] K. Ding, J. Li, R. Bhanushali, and H. Liu, “Deep anomaly detection on attributed networks,” in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2019.
- [6] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *NeurIPS*, 2017.
- [7] Y. Rong, W. Huang, T. Xu, and J. Huang, “DropEdge: Towards deep graph convolutional networks on node classification,” in *Proc. ICLR*, 2020.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *EMNLP*, 2019.
- [9] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, “Heterogeneous graph attention network,” in *WWW*, 2019.
- [10] Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” in *WWW*, 2020.
- [11] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” in *ESWC*, 2018.
- [12] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in *CIKM*, 2020.
- [13] Z. Liu, C. Chen, L. Li, J. Zhou, X. Li, L. Song, and Y. Qi, “GeniePath: Graph neural networks with adaptive receptive paths,” in *AAAI*, 2019.
- [14] M. Wang et al., “Deep graph library: A graph-centric, highly-performant package for graph neural networks,” *arXiv:1909.01315*, 2019.
- [15] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Repr. Learning on Graphs and Manifolds*, 2019.
- [16] Y. Wang, B. Feng, G. Li, S. Li, L. Deng, Y. Xie, and Y. Ding, “GNNAdvisor: An adaptive and efficient runtime system for GNN acceleration on GPUs,” in *OSDI*, 2021.